🐍 PYTHON NOTES — SETS AND MORE DATA STRUCTURES

📚 TOPIC: SETS AND MORE DATA STRUCTURES

Sets are one of Python's built-in data structures.

A set stores **unique** values and is mainly used to remove duplicates, compare collections of data, and perform mathematical set operations.

---

1. WHAT IS A SET?

A **set** is an unordered collection of **unique** items.

Unlike lists and tuples, a set **does not allow duplicate values**.

Example

```
fruits = {'apple', 'banana', 'mango'}

print(fruits)
```

Possible output:

```
{'banana', 'apple', 'mango'}
```

The order may be different because sets are unordered.

---

2. CREATING A SET

A set is created using curly braces `{}`.

Example

```
numbers = {1, 2, 3, 4}

print(numbers)
```

---

3. EMPTY SETS

Creating an empty set is different.

Incorrect

```
my_set = {}
```

This creates an **empty dictionary**, not a set.

Correct

```
my_set = set()

print(my_set)
```

Output:

```
set()
```

Important

Code	Creates
`{}`	Empty dictionary
`set()`	Empty set

---

4. SETS STORE UNIQUE VALUES

A set automatically removes duplicate values.

Example

```
fruits = {
    'apple',
    'banana',
    'apple',
    'mango',
    'banana'
}

print(fruits)
```

Possible output:

```
{'apple', 'banana', 'mango'}
```

The duplicate values disappear automatically.

---

5. SETS ARE UNORDERED

Sets do not guarantee the order of their items.

Example:

```
colors = {'red', 'blue', 'green'}

print(colors)
```

Possible outputs:

```
{'green', 'red', 'blue'}
```

or

```
{'blue', 'green', 'red'}
```

Both are correct.

Important

Do not rely on the order of items inside a set.

---

6. ADDING ITEMS — `.add()`

`.add()` adds **one** item to a set.

Example

```
fruits = {'apple', 'banana'}

fruits.add('mango')

print(fruits)
```

Adding a duplicate

```
fruits.add('apple')
```

Nothing changes because the item already exists.

---

7. ADDING MULTIPLE ITEMS — `.update()`

`.update()` adds multiple items at once.

Example

```
fruits = {'apple', 'banana'}

fruits.update(['mango', 'orange'])

print(fruits)
```

Possible output:

```
{'apple', 'banana', 'mango', 'orange'}
```

Duplicate values are ignored automatically.

---

8. REMOVING ITEMS — `.remove()`

`.remove()` removes an item from a set.

Example

```
fruits = {'apple', 'banana', 'mango'}

fruits.remove('banana')

print(fruits)
```

Output:

```
{'apple', 'mango'}
```

Important

If the item does not exist, Python raises an error.

---

9. REMOVING ITEMS SAFELY — `.discard()`

`.discard()` removes an item **only if it exists**.

Example

```
fruits = {'apple', 'banana', 'mango'}

fruits.discard('banana')
```

If the item does not exist:

```
fruits.discard('orange')
```

Nothing happens.

No error is produced.

Difference

`.remove()`	`.discard()`
Raises an error if the item is missing	Does nothing if the item is missing

---

10. REMOVING A RANDOM ITEM — `.pop()`

`.pop()` removes and returns one item from the set.

Example

```
fruits = {'apple', 'banana', 'mango'}

removed = fruits.pop()

print(removed)
print(fruits)
```

Important

Because sets are unordered, you **cannot predict** which item will be removed.

---

11. REMOVING EVERYTHING — `.clear()`

`.clear()` removes every item from a set.

Example

```
numbers = {1, 2, 3}

numbers.clear()
```

```
print(numbers)
```

Output:

```
set()
```

The set still exists.

It is simply empty.

---

12. MEMBERSHIP TESTING

We can check whether an item exists using `in` and `not in`.

Using `in`

```
fruits = {'apple', 'banana', 'mango'}

print('banana' in fruits)
```

Output:

```
True
```

Using `not in`

```
print('orange' not in fruits)
```

Output:

```
True
```

---

13. UNION

A union combines all unique items from two sets.

Example

```
A = {1, 2, 3}
B = {3, 4, 5}

print(A.union(B))
```

Output:

```
{1, 2, 3, 4, 5}
```

Shortcut

```
print(A | B)
```

---

14. INTERSECTION

An intersection keeps only the items common to both sets.

Example

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A.intersection(B))
```

Output:

```
{3, 4}
```

Shortcut

```
print(A & B)
```

15. DIFFERENCE

Difference returns the items in the first set that are **not** in the second set.

Example

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A.difference(B))
```

Output:

```
{1, 2}
```

Shortcut

```
print(A - B)
```

Important

Order matters.

```
A - B
```

is different from

```
B - A
```

16. SYMMETRIC DIFFERENCE

Symmetric difference keeps the items that appear in **only one** of the sets.

Example

```
A = {1, 2, 3}
B = {3, 4, 5}

print(A.symmetric_difference(B))
```

Output:

```
{1, 2, 4, 5}
```

Shortcut

```
print(A ^ B)
```

---

17. SUBSETS — `issubset()`

A subset means every item in one set also appears in another set.

Example

```
A = {1, 2}
B = {1, 2, 3, 4}

print(A.issubset(B))
```

Output:

```
True
```

Because every item in `A` exists in `B`.

---

18. SUPERSETS — `issuperset()`

A superset contains every item from another set.

Example

```
A = {1, 2, 3, 4}
B = {1, 2}

print(A.issuperset(B))
```

Output:

```
True
```

---

19. MUTABLE VS IMMUTABLE

Sets are **mutable**.

This means they can be changed.

Examples:

```
.add()
.update()
.remove()
.discard()
.pop()
.clear()
```

all modify a set.

Tuples are **immutable**.

Once created, tuple items cannot be changed.

---

20. CHOOSING THE RIGHT DATA STRUCTURE

Lists

Use when:

- Order matters
- Duplicates are allowed
- Items may change

Tuples

Use when:

- Order matters
- Data should not change

Dictionaries

Use when:

- Information should be stored using keys and values

Sets

Use when:

- Only unique values are needed
- Duplicate values should be removed
- Collections need to be compared
- Order does not matter

---

21. PRACTICE PROGRAM

```
students = {
    'Ben',
    'Mary',
    'Ben',
```

```
    'Chris',
    'Mary'
}

print(students)

students.add('Grace')

students.update(['Joy', 'Daniel'])

students.discard('Ben')

print(students)
```

📌 WHAT I CAN DO NOW

After learning Sets and More Data Structures, I can:

☑️ Create sets

☑️ Create empty sets

☑️ Understand that sets store unique values

☑️ Understand that sets are unordered

☑️ Add items using `.add()`

☑️ Add multiple items using `.update()`

☑️ Remove items using `.remove()`

☑️ Remove items safely using `.discard()`

☑️ Remove random items using `.pop()`

☑️ Clear a set using `.clear()`

☑️ Test membership using `in` and `not in`

☑️ Perform union operations

☑️ Perform intersection operations

☑️ Perform difference operations

☑️ Perform symmetric difference operations

☑️ Understand subsets and supersets

☑️ Choose between lists, tuples, dictionaries, and sets depending on the problem

---

🌟 MY PROGRESS

Current Level: Sets and More Data Structures ✅

Topics Completed

✅ Sets

✅ Creating sets

✅ Empty sets

✅ Unique values

✅ Unordered collections

✅ `.add()`

✅ `.update()`

✅ `.remove()`

✅ `.discard()`

✅ `.pop()`

✅ `.clear()`

✅ Membership testing

✅ Union

✅ Intersection

✅ Difference

✅ Symmetric Difference

✅ Subsets

✅ Supersets

✅ Mutable vs immutable

✅ Choosing the correct data structure

🏆 SETS COMPLETE

Sets are extremely useful whenever I need to:

- Remove duplicate values
- Compare collections of data
- Store only unique items
- Perform mathematical set operations

Although sets are not used as frequently as lists or dictionaries in beginner programs, they are very useful in real-world programming and will appear naturally in larger projects later.

🐍 SETS AND MORE DATA STRUCTURES → ✅ COMPLETE

Next Topic: 📁 FILE HANDLING